

Websockets Tests

Positive testing: Unary operations

Test 1 - FindMovie

The screenshot shows a Websocket client interface with the following components:

- URL Bar:** `ws://localhost:8080/ws/movie/detail`
- Message Input:** A text input field with the value `1`.
- Response Panel:** Displays a JSON response: `{ "id": 1, "title": "Inception", "releaseYear": 2010, "runtimeMinutes": 148, "director": "Christopher Nolan", "synopsis": "A thief who steals corporate secrets through dream-sharing tec..." }`
- Status:** A green "Connected" indicator is visible.
- Timestamps:** The response is timestamped at `15:57:39.716`.

- A valid **movie_id: 1** was sent to the server. And then the server successfully established the Websocket connection and returned a response containing the full movie details for inception.
- This demonstrates that the Unary operation correctly fetches data from the SQLite database and returns the result as JSON through the WebSocket connection.

Negative testing: Unary operations

The screenshot shows a Websocket client interface with the URL `ws://localhost:8080/ws/movie/detail`. The message input field contains the text `abc`. The interface includes tabs for Docs, Message, Params, Headers, and Settings. A 'Send' button is located at the bottom right. The 'Response' section shows a 'Disconnected' status. The message history includes:

- Disconnected from `ws://localhost:8080/ws/movie/detail` at 16:08:35.319
- Fejl i Film ID format. Vær sikker på at du sender et gyldigt heltal. at 16:08:35.319
- abc at 16:08:35.316
- Connected to `ws://localhost:8080/ws/movie/detail` at 16:08:34.359

Test 2 - FindMovie (Invalid Characters)

- Sent a invalid character **movie_id: abc** to the server instead of a valid integer value. And the server responded with: *"Fejl i Film ID format. Vær sikker på at du sender et gyldigt heltal."*
- This demonstrates that the WebSocket service correctly validates the input before processing the request and handles invalid data gracefully by returning a custom error message to the client.

The screenshot shows a Websocket client interface with the URL `ws://localhost:8080/ws/movie/detail`. The message input field contains the text `99`. The interface includes tabs for Docs, Message, Params, Headers, and Settings. A 'Send' button is located at the bottom right. The 'Response' section shows a 'Disconnected' status. The message history includes:

- Disconnected from `ws://localhost:8080/ws/movie/detail` at 16:14:25.789
- Filmen med id99 blev ikke fundet at 16:14:25.789
- 99 at 16:14:25.781
- Connected to `ws://localhost:8080/ws/movie/detail` at 16:14:24.941

Test 3 - FindMovie (Not Found)

- Sent a valid character **movie_id: 99** that does not exist in the database to the server.
- The server then responded with a custom error message indicating that the movie could not be found: *"Fejl i Film ID format. Vær sikker på at du sender et gyldigt heltal."*
- This demonstrates that the WebSocket service correctly handles requests for non-existing resources and gracefully returns an appropriate error response to the client.

Positive testing: Bidirectional Streaming operations

The screenshot shows a web browser window with the title "Stream Film - Positiv Test". The address bar shows the URL "ws://localhost:8080/ws/movie/stream". Below the address bar, there are tabs for "Docs", "Message", "Params", "Headers", and "Settings". The "Message" tab is active, showing a list of messages. The messages are in JSON format, containing movie data. The list includes movies like Blade Runner, The Godfather, Alien, Forrest Gump, Parasite, The Matrix, Pulp Fiction, Interstellar, The Dark Knight, and Inception. The connection status is "Connected".

Message ID	Movie Data	Timestamp
10	{"id":10,"title":"Blade Runner 2049","releaseYear":2017,"runtimeMinutes":164,"director":"Denis Villeneuve","synopsis":"A young blade runner's discovery of a long-buried secr..."}	16:18:27.157
9	{"id":9,"title":"The Godfather","releaseYear":1972,"runtimeMinutes":175,"director":"Francis Ford Coppola","synopsis":"The aging patriarch of an organized crime dynasty trans..."}	16:18:25.156
8	{"id":8,"title":"Alien","releaseYear":1979,"runtimeMinutes":117,"director":"Ridley Scott","synopsis":"After a space merchant vessel receives an unknown transmission as a dis..."}	16:18:23.152
7	{"id":7,"title":"Forrest Gump","releaseYear":1994,"runtimeMinutes":142,"director":"Robert Zemeckis","synopsis":"The presidencies of Kennedy and Johnson, the Vietnam War, the..."}	16:18:21.146
6	{"id":6,"title":"Parasite","releaseYear":2019,"runtimeMinutes":132,"director":"Bong Joon-ho","synopsis":"Greed and class discrimination threaten the newly formed symbiotic r..."}	16:18:19.145
5	{"id":5,"title":"The Matrix","releaseYear":1999,"runtimeMinutes":136,"director":"Lana Wachowski, Lilly Wachowski","synopsis":"A computer hacker learns from mysterious rebels..."}	16:18:17.142
4	{"id":4,"title":"Pulp Fiction","releaseYear":1994,"runtimeMinutes":154,"director":"Quentin Tarantino","synopsis":"The lives of two mob hitmen, a boxer, a gangster and his wi..."}	16:18:15.140
3	{"id":3,"title":"Interstellar","releaseYear":2014,"runtimeMinutes":169,"director":"Christopher Nolan","synopsis":"A team of explorers travel through a wormhole in space in a..."}	16:18:13.129
2	{"id":2,"title":"The Dark Knight","releaseYear":2008,"runtimeMinutes":152,"director":"Christopher Nolan","synopsis":"When the menace known as the Joker wreaks havoc and chao..."}	16:18:11.127
1	{"id":1,"title":"Inception","releaseYear":2010,"runtimeMinutes":148,"director":"Christopher Nolan","synopsis":"A thief who steals corporate secrets through dream-sharing tec..."}	16:18:09.021

The connection status is "Connected" and the URL is "ws://localhost:8080/ws/movie/stream".

Test 4 - FindMovie

- A valid request: **1** was sent to the server. The server successfully established the WebSocket connection and continuously streamed movie data back to the client every 2 seconds.
- The response contained multiple movie objects from the SQLite database returned as JSON through the WebSocket connection.
- This demonstrates that the Bidirectional Streaming operation correctly keeps the connection open and continuously streams data between the server and client in real time.

Negative testing: Bidirectional Streaming operations

The screenshot shows a WebSocket client interface titled "Stream Film - Negativ Test - Ugyldig". The URL bar contains "ws://localhost:8080/ws/movie/stream". The "Message" tab is active, showing a single message with the text "abc". The "Response" section shows a "Connected" status and a message: "Fejl i Film ID format. Vær sikker på at du sender et gyldigt heltal." (Error in Film ID format. Be sure you send a valid integer). The message is timestamped 16:20:17.505. Below the message, there is a "Send" button and a "Saved messages" list.

Test 5 - FindMovie (Invalid Characters)

- Sent invalid characters: **abc** to the server instead of a valid integer value. The server responded with a custom error message indicating that the request format was invalid: *"Fejl i Film ID format. Vær sikker på at du sender et gyldigt heltal."*
- This demonstrates that the WebSocket service correctly validates the input before processing the streaming request and gracefully handles invalid data by returning an appropriate error message to the client.

The screenshot shows a WebSocket client interface titled "Stream Film - Negativ Test - Ikke fundet". The URL bar contains "ws://localhost:8080/ws/movie/stream". The "Message" tab is active, showing a single message with the text "99". The "Response" section shows a "Connected" status and a message: "Film med id 99 blev ikke fundet" (Movie with id 99 was not found). The message is timestamped 16:22:04.020. Below the message, there is a "Send" button and a "Saved messages" list.

Test 6 - FindMovie (Not Found)

- Sent a valid movie_id: 99 that does not exist in the database to the server. The server then responded with a custom error message indicating that the requested movie could not be found: *"Film med id 99 blev ikke fundet"*
- This demonstrates that the WebSocket service correctly handles requests for non-existing resources during streaming operations and gracefully returns an appropriate error response to the client.